

Nama : Muh Khalifman Ali

NIM : 245150407111079

Matkul : Pengembangan Aplikasi Mobile - D

Tugas Modul 4 Praktikum

I. State Hoisting

A. Aktivitas percobaan

- Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.

1. Apa yang terjadi setiap kali tombol "Increment" diklik?

Angka di tampilan text bertambah 1 dengan berurutan (0, 1, 2, 3, dst)

2. Apa yang menyebabkan hal tersebut terjadi?

Klik pada tombol memicu blok `onClick = { number++ }` yang mengubah nilai state `number`. Perubahan state tersebut terdeteksi oleh Compose dan memicu proses recomposition pada fungsi Composable `CounterScreen`.

3. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Composable Text (Angka): Variabel `number` (`var number by remember { mutableStateOf(0) }`).
Composable Button (Label Teks): Variabel `label` (`val label by remember { mutableStateOf("Increment") }`).

4. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state `label` (`val label by remember { mutableStateOf("Increment") }`).

- Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

1. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan dilakukan di dalam body fungsi Composable CounterScreen:

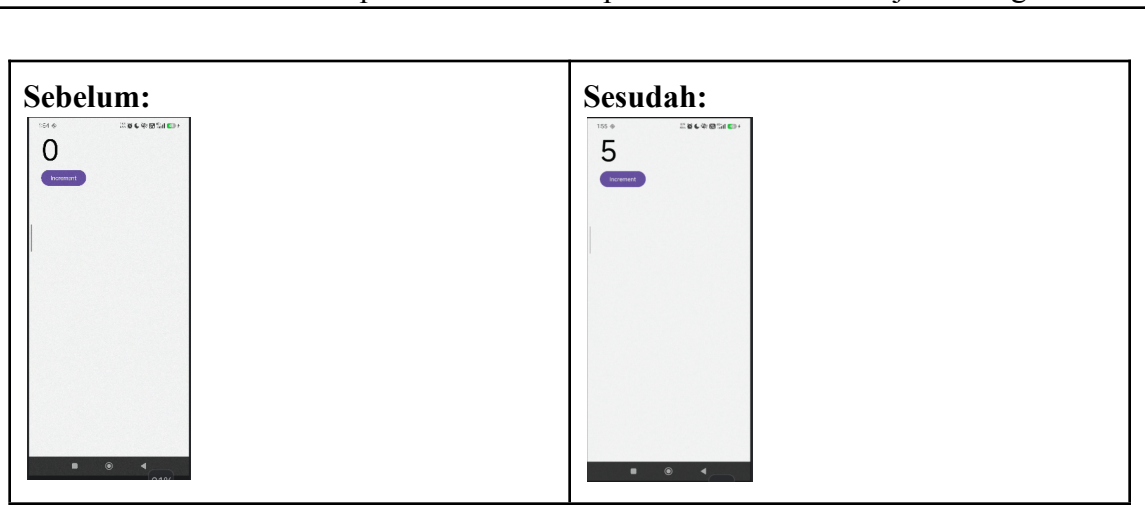
- a). Mengubah inisialisasi state label menjadi "Increment 2".
- b). Mengubah logika operasi di dalam lambda onClick tombol dari `number++` menjadi `number += 2`.

Kode:

```
Kotlin
@Composable
fun CounterScreen(modifier: Modifier) {
    var number by remember { mutableStateOf(0) }
    val label by remember { mutableStateOf("Increment 2") } //
    Perubahan 1: Label

    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = { number += 2 }) { // Perubahan 2:
    Increment +2
        Text(text = "$label")
    }
    }
}
```

2. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



- Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

1. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

iya program berjalan dengan baik tanpa error dan memberikan perilaku UI yang sama persis.

2. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment dilakukan di fungsi onCreate() (fungsi pemanggil/parent), lokasinya terdapat di callback onClick = { number++ }.

3. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State number dideklarasikan di dalam fungsi onCreate() menggunakan var number by remember { mutableStateOf(0) }. Sedangkan label tombol "Increment" dideklarasikan dan dikirim sebagai argumen String langsung dari fungsi onCreate().

- Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

1. Label tombol "Increment" menjadi "Double"

```
label = "Double",
```

2. Nilai awal variabel number dari 0 menjadi 1.

```
var number by remember { mutableStateOf( value = 1) }
```

3. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.

```
onButtonClick = {
    number *= 2
}
```

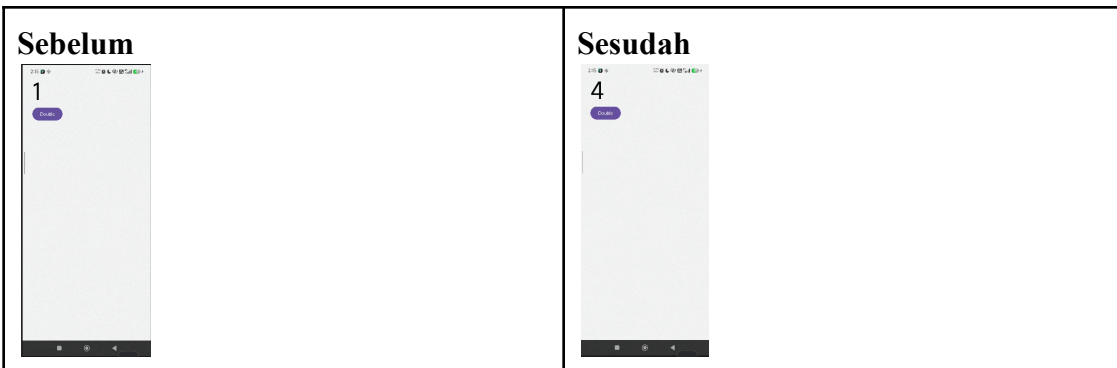
4. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
Kotlin
setContent {
    var number by remember { mutableStateOf(1) }

    CounterScreen(
        modifier = Modifier.padding(32.dp),
        number = number,
        label = "Double",
        onButtonClick = {
            number *= 2
        }
    )
}
```

- Jalankan program.

1. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



-
2. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Seluruh perubahan (mengubah nilai awal state, merubah label tombol, dan mengganti logika dari penambahan menjadi perkalian) hanya dilakukan pada fungsi

3. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan sama sekali pada fungsi Composable CounterScreen. Kode CounterScreen tetap menggunakan struktur Stateless yang sama seperti pada tahap 7.

4. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

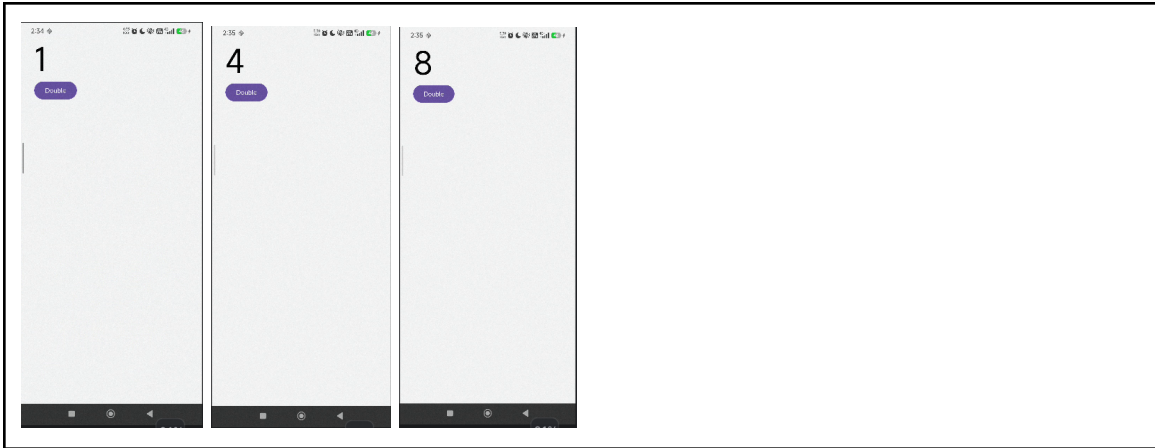
Hal ini membuktikan manfaat utama dari State Hoisting:

1. Reusability & Statelessness: Komponen UI (CounterScreen) menjadi bersifat stateless dan sangat fleksibel untuk digunakan kembali (reusable).
2. Separation of Concerns & SSoT: Memisahkan logika bisnis (manajemen state) dari logika presentasi UI. Ketika terjadi perubahan fungsionalitas (seperti mengubah tombol dari increment menjadi penggandaan angka), pengembang cukup mengubah logika pada komponen induk (onCreate) tanpa perlu menyentuh atau merusak komponen UI anak (CounterScreen).

II. Perubahan konfigurasi

A. Aktivitas percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



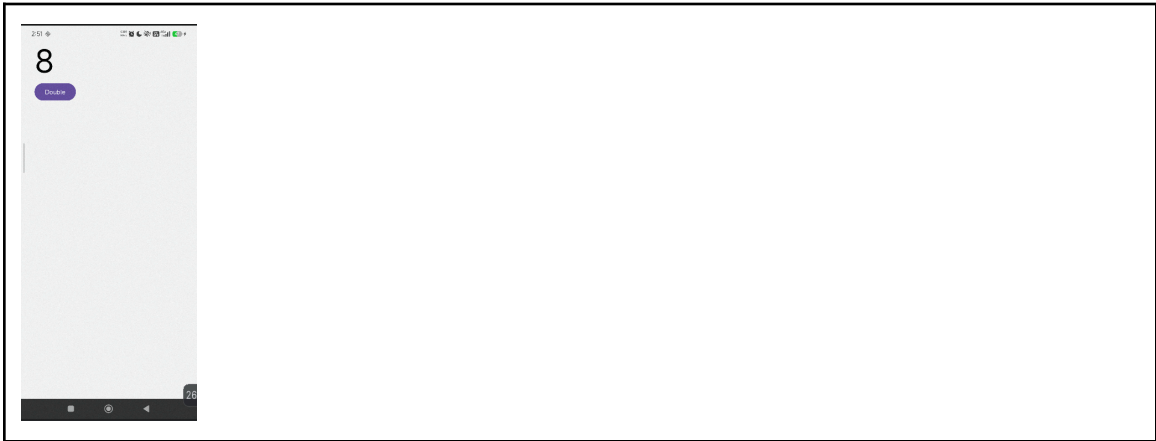
- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan: 1

Alasan mengapa: Saat posisi perangkat diputar dari portrait ke landscape, Android melakukan Configuration Change (Perubahan Konfigurasi) yang menghancurkan (destroy) Activity saat ini dan membuatnya kembali (recreate). Konstruk remember hanya mampu menyimpan nilai state selama proses recomposition saat Activity masih hidup. Karena Activity dihancurkan dan dibuat ulang akibat rotasi, seluruh memori remember terhapus sehingga variabel number terinisialisasi ulang ke nilai awalnya, yaitu

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel

number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang tampil: 8

Alasannya: Konstruk `rememberSaveable` memanfaatkan mekanisme Saved Instance State (Bundle) milik sistem Android. Ketika Activity dihancurkan dan dibuat ulang akibat perubahan konfigurasi (rotasi layar), Compose akan memulihkan (restore) nilai number dari Bundle tersebut sehingga nilainya tidak kembali ter-reset ke angka awal 1.

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

remember: Hanya menyimpan state selama siklus recomposition dalam satu

instance Activity yang sama. Nilai akan hilang jika Activity dihancurkan dan dibuat ulang (recreate) akibat perubahan konfigurasi (seperti rotasi layar, perubahan bahasa, atau pergantian tema).

rememberSaveable: Selain menyimpan state saat recomposition, ia juga menyimpan nilai ke dalam Bundle (Saved Instance State) sehingga state tetap bertahan menembus Perubahan Konfigurasi (Configuration Change) maupun pemusnahan proses sementara oleh sistem.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh Kasus Nyata:

1. Input Form / Isian Teks: Menyimpan teks yang sedang diketik pengguna pada `OutlinedTextField` (seperti nama, email, atau alamat) saat layar tidak sengaja terputar.
2. Filter & Checkbox: Menyimpan status centang (checked/unchecked) pada pilihan filter pencarian produk e-commerce.
3. Posisi Tab Navigasi: Menyimpan indeks halaman/tab yang sedang aktif agar tidak kembali ke Tab pertama saat rotasi.

Manfaat yang Diperoleh:

1. Meningkatkan User Experience (UX): Pengguna tidak perlu mengulang mengetik atau memilih data dari awal ketika terjadi rotasi layar secara tidak sengaja.
2. Menjaga Integritas Data Sementara: Data masukan pengguna di layer UI tetap aman sebelum dikirimkan ke database/server.

III. Launched Effect

A. Aktivitas percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Di layar HP muncul angka 1, tombol "Double", dan di bagian bawah layar menampilkan: "Number 1 is shown!"

b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena fungsi `LaunchedEffect` dipanggil paling pertama dan karena variabel `number` terinisialisasi dengan nilai 1, `LaunchedEffect(number)` langsung mengeksekusi blok kode itu untuk menampilkan notif tanpa menunggu tombol diklik.

c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicu pesan `SnackBar` adalah pemanggilan fungsi `suspend` `snackbarHostState.showSnackBar(...)`.

Kotlin

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackBar("Number $number is shown!")  
}
```

2. Klik button "Double" satu kali,

a. Apa yang terjadi?

Angka di layar berubah dari 1 menjadi 2, kemudian `SnackBar` lama menghilang dan digantikan oleh `SnackBar` baru bertuliskan:

"Number 2 is shown!"

b. Mengapa hal tersebut terjadi? Jelaskan!

Hal ini terjadi karena variabel `number` merupakan key dari `LaunchedEffect(number)`. Ketika tombol "Double" diklik, nilai `number` berubah (menjadi 2). Perubahan nilai key ini mendeteksi bahwa state telah berubah, sehingga `LaunchedEffect` secara otomatis membatalkan coroutine yang lama dan menjalankan ulang (re-launch) blok kode di dalamnya dengan membawa nilai `number` terbaru (2).

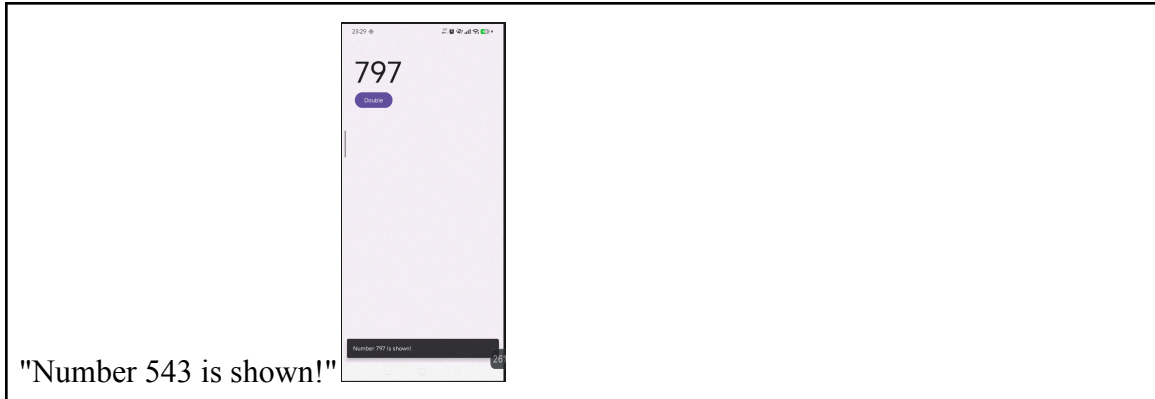
3. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Di layar HP muncul angka 1, tombol "Double", dan di bagian bawah layar menampilkan: "Number 1 is shown!"

b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah jeda beberapa detik, angka 1 di layar mendadak berubah menjadi angka acak antara 100 sampai 1000 (misal pada layar: 797), dan notif ikut diperbarui menampilkan



- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Yang mengtrigger perubahan adalah fungsi `Repo.getData()` yang memberikan angka acak dan memperbarui nilai number. Ketika number berubah, `LaunchedEffect(number)` pertama ikut kettrigger untuk memunculkan notif baru.

Bagian kode program yang mengtrigger:

```
Kotlin
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

4. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)`: Menggunakan variabel state (seperti number). Blok kode di dalamnya akan dieksekusi paling pertama dan akan dieksekusi ulang setiap kali nilai state mengalami perubahan.

`LaunchedEffect(Unit)`: Menggunakan konstanta `Unit` yang nilainya tidak pernah berubah sebagai key. akhirnya, blok kode di dalamnya hanya akan dieksekusi satu kali saja saat Composable pertama kali masuk ke dalam komposisi (mount).

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

`LaunchedEffect(state)`: Dijalankan pada awal komposisi pertama dan dijalankan ulang setiap kali nilai state (key) berubah cocok digunakan untuk aksi yang bergantung pada data UI (seperti menampilkan Snackbar atau validasi masukan saat data berubah).

`LaunchedEffect(Unit)`: Dijalankan hanya sekali saja di awal saat layar dibuka. cocok

digunakan untuk inisialisasi awal seperti fetch API/data internet, memuat data dari database lokal saat layar pertama kali dimuat, atau mengtrigger animasi awal.